Automated Breadboard Circuit Verification using Heading-Aware Deep Reinforcement Learning

Viktor Trilar
Turku University of Applied Sciences, Finland
(Research Exchange, NIT Sendai College, Japan)
Supervisor: Jun Suzuki
National Institute of Technology, Sendai College, Japan

1. Introduction

Circuit assembly is one of the core practical skills that students learn in electronics. Circuits are built by inserting components and wires into a breadboard, and correctness is then checked by measuring electrical behavior. Before any measurement is meaningful, the wiring itself has to be verified, which currently requires a human instructor. As class sizes grow, this becomes a bottleneck that slows student learning.

This paper presents an automated circuit verification system developed under JSPS Grant No. 24K06211. Given one smartphone photo of a student's breadboard, the system reads off the wiring topology and compares it against a reference circuit. An instructor is not needed. The starting point is Takahashi [1], who built the first wire tracing pipeline at NIT Sendai using CIELAB color segmentation and a Deep Q-Network (DQN) tracer, reaching 96.3% wire match rate on synthetic data. Three things were left undone: handling identical color wire crossings, detecting components, and deploying the system for classroom use. All three gaps are addressed here, along with two new geometry methods for arcing wires and blind endpoint recovery.

Caporali et al. [3] introduced Ariadne+, a framework that separates overlapping wires through a superpixel graph walk without requiring known endpoints. It performs well on thick industrial wiring but the authors note a failure case directly relevant here: when two wires are similar in color and curvature, the graph walk can jump between them at the intersection point. The breadboard adds another constraint: wires must terminate at specific holes in a regular grid rather than follow free form paths through space.

Mnih et al. [2] showed that a neural network trained with experience replay and a target network can learn navigation directly from pixel observations. Mirowski et al. [6] extended this to visually complex 3D environments, finding that auxiliary goal representations improve efficiency. The same idea is adapted here to a flat 2D mask, where the difficulty is not avoiding a barrier but choosing between two visually similar arms at the intersection point.

Glučina et al. [5] showed that YOLO detectors classify PCB components well under controlled industrial imaging. Breadboard photos are different: components are placed by hand, unlabeled, and captured under variable lighting and angles with different camera hardware. A model trained on PCB data needs domain specific fine tuning to work on breadboards.

Takahashi [1] used CIELAB thresholding with morphological filtering and skeletonization for wire masks, then trained a DQN with a 10-dimensional forward view observation: a 2×5 grid of pixel values sampled ahead of the agent, with five discrete actions. Among three observation designs, the forward view approach reached 95.8% path progress and was selected. The final wire match rate was 96.3% on synthetic data. Identical color crossings, component detection, validation, and deployment were not addressed.

2. Experimental Methods

The system runs a photograph through five stages. Figure 1 shows the full pipeline.

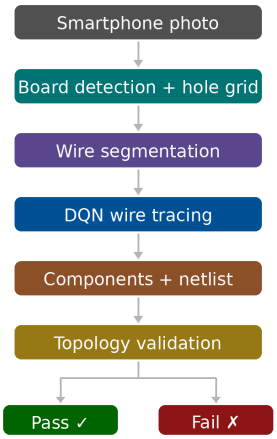

Figure 1: Pipeline for automated breadboard circuit verification.

A YOLOv8 pose model finds the four board corners and projects a hole grid into the image (300 holes for a 10-row board, 480 for a 12-row). The board type is read from the same pose output. Two candidate grids are scored against the actual hole positions: a parametric

projection from the corners and a homography fit from holes detected by YOLO. The one with a higher score on a given image is used.

Distinct color wires are segmented using CIELAB thresholding. Power rails and component bounding boxes are excluded from the wire mask to prevent false detections. White and other neutral wires need a separate path because they look too similar to the board surface using CIELAB space. For these, a YOLOv8 instance segmentation model produces per wire masks. An important detail is that the DQN is trained on the same type of masks it sees at inference time. Earlier versions used cleaner training masks and ran into near miss navigation failures when the noisier deployment masks did not match.

The DQN navigates from a starting hole to an ending hole on the wire mask. The observation is a 14-dimensional vector (Figure 2):

- 10 forward view wire signals: pixel values on a 2×5 grid oriented along the heading
- $\sin\theta$ and $\cos\theta$ of the current heading θ.
- Normalized Euclidean distance to the goal.
- On wire signal at the current position.

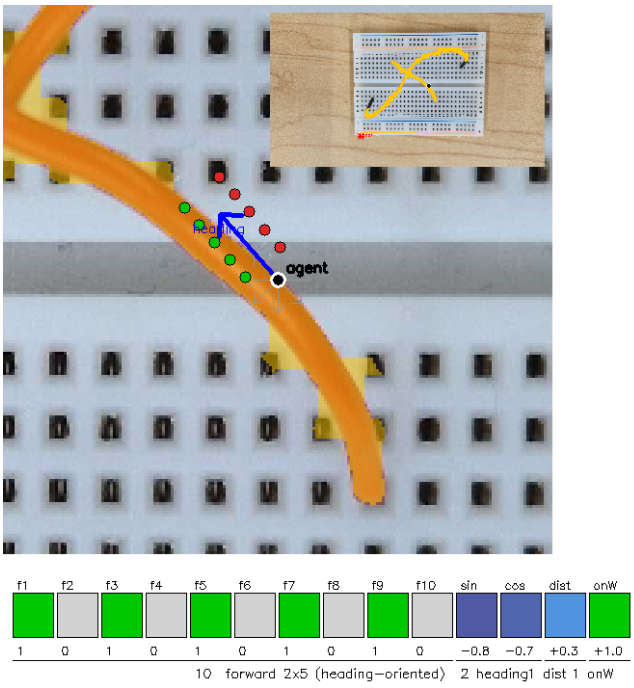

Figure 2: Agent state at a wire crossing. The 2×5 forward view grid samples the mask ahead of the agent (green = on wire, red = off wire). The heading arrow (blue) and eight compass actions are shown. Bottom: the 14-dimensional state vector with values computed at this position. Inset: full board context.

The addition of heading information is what makes the disambiguation for wire crossing work. At an identical color intersection point the local mask looks roughly the same in both directions, so the original 10-dimensional signal gives no basis for choosing which arm to follow. With $(\sin\theta, \cos\theta)$ in the observation, the agent can prefer the arm whose direction matches its current trajectory. Eight compass direction actions replace the original five, which helps on diagonal wires.

When the DQN's path on wire score drops below 0.5, an arc fitting fallback takes over. A set of parabolic arcs with varying sag is fitted between the two endpoints, and the arc with the highest mask coverage wins. The system keeps $\max(\text{DQN}, \text{arc fit})$, so a cleanly traced flat wire is never overridden. The fallback handles arcing jumper wires, where the wire physically lifts off the board surface and the grid space DQN picks up almost no on wire signal along the curve.

When endpoints are not known ahead of time, the system recovers them geometrically. For every wire, the system runs PCA on the mask to find the principal axis, fits a curve through the pixel distribution, and reads off the two ends. The method is fragment robust: it returns the same endpoints whether the mask is fully connected, broken into pieces, or reduced to just two end stubs, which matters for arcing wires whose masks can break apart as the wire rises away from the board surface and where connected component analysis fails entirely.

A YOLOv8 detector fine tuned on 40 labeled real photographs identifies four classes: LED, resistor, transistor, and capacitor. Geometric pin offset rules turn bounding boxes into hole assignments. A union-find algorithm then merges wire connections and component pins into electrical nets. Validation is by connectivity, not by position: a circuit built in a different position passes if it is electrically equivalent to the reference.

The pipeline runs inside a bilingual Japanese/English mobile first FastAPI application. A student takes a photo, picks the exercise to compare against, and receives an annotated pass/fail verdict within one second, with each component and wire labeled at its specific hole. Preflight checks for board detection confidence and blur and rejects unusable submissions before GPU inference. Instructors create exercises through an interactive labeling tool, and a teacher dashboard tracks pass rates and common errors per exercise with CSV export. Figure 3 shows the student view.

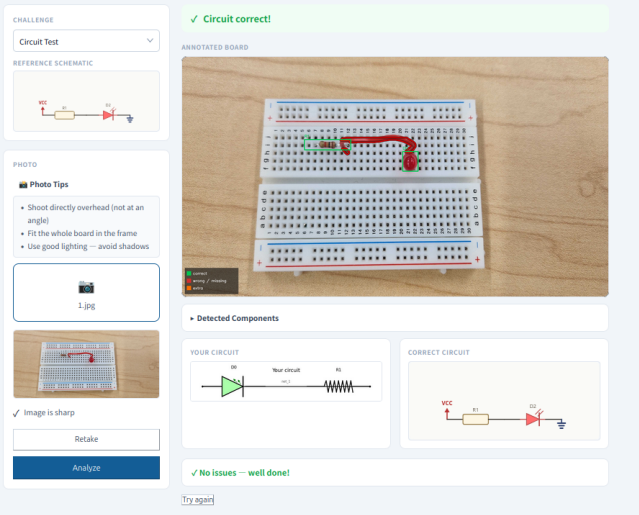

Figure 3: Student submission view showing the annotated board with per hole locations and a pass/fail verdict.

3. Results and Discussion

Table 1 collects all performance numbers. The results are split by evaluation type.

Table 1: System performance summary. GT-fed = ground truth endpoints provided; Blind = no prior knowledge of wire positions.

Metric	Value	Sample size	Evaluation
Wire match, synthetic	99.4%	3,000 images	GT-fed
Identical color crossing, synthetic	99.5%	crossing pairs	GT-fed
Distinct color wires, real	100% (65/65)	10 boards	GT-fed
Navigation failures	6, all neutral	96 evaluable	Segmentation
Arc fit on arcing wires	0.50–0.66	path on wire	GT-fed
Board level wire discovery	87% (45/52)	52 images	Blind
Board level, distinct color, flat wires	≈ 100%	40 images	Blind
Component detection (mAP@50)	99.5%	4 classes	Real images

Training and evaluation used 3,000 Blender rendered images. The 14-dimensional agent reaches 99.4% wire match rate, up from Takahashi's 96.3% (+3.1 pp). On identical color crossings the disambiguation accuracy is 99.5%, a case that the original formulation could not handle.

Across 10 real boards, all 65 distinct color wires (red, blue, green, orange, purple, brown, yellow) matched correctly (100%), including identical color crossings with no arm swaps. Match rate here means the model reached the correct goal hole. Path on wire measures whether the model followed the wire rather than taking a shortcut through open space; it is reported separately for arcing wires below.

The same evaluation covered 31 neutral wires (white and black), of which 25 nominally passed. Those results are not meaningful: CIELAB cannot distinguish neutral insulation from the breadboard surface, so the mask ends up covering most of the image. With no real wire signal to follow, navigation collapses into goal seeking on a nearly uniform mask, and any apparent passes are coincidental. All 6 navigation failures fall into this category. There were zero distinct color failures.

On arcing wires where the DQN scored 0.01–0.12 on path on wire, arc fitting raised coverage to 0.50–0.66 and crossed the navigation threshold in every case. On flat wires with clean masks the DQN score dominates and the fallback does not trigger, so the system degrades gracefully. No retraining was needed.

The endpoint recovery algorithm was accurate to within 7 px on synthetic arcing wires across three mask conditions: fully connected, broken into four pieces and reduced to end stubs only, with identical recovery in all three cases. On real images the algorithm recovered endpoints for isolated distinct color wires whenever the per wire segmentation produced a clean mask. The limiting factor for broader real image performance is segmentation quality, not the algorithm itself.

Across 52 breadboard test images without ground truths, 45 found at least one wire (87% board level) after the arc fit confirmation was integrated. The 7 remaining misses are all neutral circuits. For distinct color boards, board level discovery is essentially 100%. Wire level blind recall is not reported because the test set has no per wire annotations, and blind discovery produces both misses and false positives that a simple found/present ratio would conflate.

Fine tuned on 40 real photographs, the detector reaches 99.5% mAP@50 across four classes. A known weakness shows up for resistors that are small in frame or partially occluded: the detector returns a body only bounding box at confidence just below threshold, which the shape filter then rejects as too square. The weakness traces to training data coverage, not model architecture.

Table 2 shows the key differences from Takahashi's baseline.

Table 2: Comparison with Takahashi (2025)

Aspect	Takahashi	This work
Obs. dims	10	14
Actions	5	8 (compass)
White wires	CIELAB only	YOLO-seg
Crossings	Not addressed	100% (distinct-col.)
Match (synth.)	96.3%	99.4%
Real image	Not reported	100% (GT, dist.-col.)
Arc navigation	Not addressed	0.66 (arc fit)
Blind endpoints	Not addressed	Fragment robust (7 px)
Components	None	4 classes
Validation	None	Net based
Deployment	None	Mobile webapp

Several limitations remain open for future work. White and black wires both hit the same wall: CIELAB cannot separate them from the board background, so the mask ends up covering most of the image. The YOLO segmentation path for white wires generates 4–7 false positive detections per board when enabled, making it unusable in practice. The path forward is a larger annotated dataset of neutral wires under varied lighting and precision focused retraining. Crossing disambiguation for these colors cannot be tested until segmentation is solved.

The discover_endpoints algorithm has been validated in isolation. The clear next step is to feed wire_tracer's per wire masks, which already exclude rails, LED bodies, and other wires, into discover_endpoints as the endpoint finding stage, replacing the current skeleton tip method. The combination pairs proven endpoint recovery with the pipeline's mature segmentation and should extend blind arcing wire discovery to real photos.

Arc fitting separates distinct color arcing wires cleanly, but for tight identical color arcing crossings the mask is one dense blob and the TRUE and SWAP arc coverages overlap with no clean separation. Solving this case likely requires either per instance mask separation, such as SAM based prompting from endpoints, or explicit occlusion ordering at the crossing.

The resistor detector returns body only bounding boxes at confidence below the threshold when the component is small or partially occluded, and the shape filter rejects them. The proper fix is retraining on a dataset that includes small, distant, and partially occluded resistors. An EXIF and image normalization rotation inconsistency is a likely contributing factor and should be resolved first.

Replacing the hand engineered 14-dimensional observation with a small attention module over the raw 2×5 forward view patch could learn richer spatial representations without needing the explicit heading encoding. A transformer based Q-network over recent observation history would also give the agent trajectory memory at intersection points.

The row half label assignment is geometrically ambiguous and is further complicated by EXIF and normalization rotations. Reading printed board edge labels via OCR or a vision language model could be a future solution. Currently, hole labels match the annotated image shown to the student but may differ from the board's printed letters. The mismatch is a display only issue.

Ten boards from one lab is a thin basis for generalization claims. A dataset of 30–50 boards from multiple photographers and lighting conditions is the right next step. The component detector should also be extended to cover other components such as buttons, potentiometers, and integrated circuits.

4. Conclusion

The heading aware DQN introduced here resolves identical color wire crossing disambiguation for distinct color configurations, the main open problem in Takahashi's baseline. Adding heading encoding, goal distance, and an on wire signal to the 10-dimensional forward view observation breaks the symmetry that previously caused arm swaps at intersection points. Synthetic wire match rises from 96.3% to 99.4%, and on 65 distinct color real image wires the system reaches 100% with ground truth endpoints provided. Two geometry primitives introduced without any retraining extend the system to arcing wires: a parabolic arc fit fallback that raised arcing wire path on wire from 0.01–0.12 to 0.50–0.66, and a fragment robust endpoint recovery algorithm that locates wire endpoints correctly even from partial or broken masks. Blind board level wire discovery reaches 87% across 52 test images, with essentially 100% on boards with flat, distinct color wires. Every navigation failure traces back to one root cause: CIELAB segmentation cannot separate neutral wire colors from the board background. Component detection (99.5% mAP@50), net based validation, and a bilingual classroom web application round out the system and make it ready for beta use.

5. Acknowledgments

This work was supported by JSPS KAKENHI Grant Number JP 24K06211.

References

[1] Y. Takahashi, "Development of an Educational Support System Using Image Recognition," Graduation Thesis, NIT Sendai College, 2025.

[2] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[3] A. Caporali, R. Zanella, D. De Gregorio, and G. Palli, "Ariadne+: Deep Learning-based Augmented Framework for the Instance Segmentation of Wires," *IEEE Trans. Ind. Informatics*, vol. 18, no. 12, pp. 8607–8617, 2022.

[4] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. `https://github.com/ultralytics/ultralytics`

[5] M. Glučina, N. Anđelić, I. Lorencin, and Z. Car, "Detection and Classification of Printed Circuit Boards Using YOLO Algorithm," *Electronics*, vol. 12, no. 3, p. 667, 2023.

[6] P. Mirowski et al., "Learning to Navigate in Complex Environments," *arXiv:1611.03673*, 2016.